Smart EB Connect: Location-Based Electricity Fault Reporting & Technician Assistance Web Application

Full-Stack Development (FSD) Final Project Report & Technical Documentation

Technology Stack: React.js (Vite) • Node.js + Express.js • MongoDB (Mongoose) • Leaflet + OpenStreetMap

---

1. Abstract

Smart EB Connect is a full-stack web application designed to streamline the reporting, geographic matching, and resolution tracking of residential and commercial electricity faults. Built using React.js, Node.js, Express.js, and MongoDB, the system supports three distinct role-based experiences: Customers, Technicians, and Administrators.

Customers can log electricity fault complaints, optionally share consented browser GPS coordinates (or fall back to a manual street address), discover available technicians ranked by straight-line Haversine distance, and monitor real-time status updates. Technicians manage their availability, update their active field coordinates, accept or reject service assignments with server-side concurrency guards, and progress complaints through a validated state machine (pending → assigned → accepted → on_the_way → in_progress → resolved). Administrators oversee platform health, manage user accounts, and assign or reassign service requests.

As an academic software-only prototype, the platform enforces strict safety boundaries—alerting users to stay clear of hazardous incidents (such as sparking, exposed wires, or burning smells) and directing them to official utility emergency helplines (1912 / 112).

---

2. Problem Statement & Objectives

2.1 Problem Statement

Traditional electricity fault reporting often suffers from opaque status tracking, inaccurate location descriptions, manual dispatch delays, and lack of visibility between consumers and field technicians. Consumers frequently do not know whether their complaint has been acknowledged or when a technician will arrive, while field technicians lack structured geographic context and clean workflow state management.

2.2 Project Objectives

- **Three-Tier Full-Stack Architecture:** Demonstrate clean separation of concerns across a React SPA frontend, a stateless Express/Node.js REST API, and a persistent MongoDB document store.
- **Role-Based Access Control (RBAC):** Enforce strict server-side authentication and authorization across `customer`, `technician`, and `admin` roles using JWT and bcrypt password hashing.
- **Geolocation & Distance Ranking:** Capture consented browser GPS coordinates (with graceful fallback to manual addresses or map selection) and rank available technicians using the Haversine great-circle formula while clearly communicating that straight-line distance differs from road travel distance/ETA.
- **Auditable Complaint Lifecycle:** Enforce valid state transitions (`pending → assigned → accepted → on_the_way → in_progress → resolved` plus `rejected` and `cancelled`) and maintain an immutable `statusHistory` audit trail on every complaint.
- **Electrical Safety & Scope Boundaries:** Operate strictly as a software-only coordination prototype (no ESP32/hardware sensors or unauthorized utility dispatch claims) with automated hazard warnings for life-safety events.

---

3. Literature & Background Survey

Modern civic and utility service platforms rely on geospatial indexing and stateful ticket workflows:

- **Geospatial Coordinate Standards (RFC 7946 GeoJSON):** MongoDB `2dsphere` indexes require coordinates in `[longitude, latitude]` order, whereas the

W3C Browser Geolocation API returns `{ latitude, longitude }`. Careful coordinate normalization at the API boundary prevents inverted spatial queries.

- **Haversine Formula vs. Road Routing:** Straight-line spherical distance provides fast, zero-cost proximity ranking suitable for academic prototypes without requiring paid commercial routing API keys, provided the user interface explicitly notes that road travel distance and arrival time may vary.
- **Role-Based Access Control & Auditability:** Soft-deletion (`cancelled` state) preserves historical operational records for utility auditing, while server-side role middleware prevents horizontal privilege escalation (e.g., one customer reading another customer's complaint or a technician modifying an unassigned ticket).

---

4. System Architecture & Request Flow

4.1 End-to-End Request Flow

`React UI (Vite + React Router + Leaflet)` → `HTTP JSON Request with Bearer JWT` → `Express Route & Security Middleware (Helmet, CORS, Auth, RBAC, Zod Validation)` → `Controller & Haversine/Workflow Logic` → `Mongoose ODM Schema` → `MongoDB Community Server (arm64)` → `Standardized JSON Response ({ success, data })` → `React State Update`.

4.2 Backend Layered Organization

- `server/src/config/`: Environment configuration and resilient MongoDB connection handler.
- `server/src/models/`: Mongoose schemas (`User`, `TechnicianProfile`, `Complaint`) with `2dsphere` geospatial indexes.
- `server/src/middleware/`: JWT verification (`authenticate`), role enforcement (`requireRole`), database readiness check (`ensureDbConnected`), Zod schema validation (`validateBody`), and centralized error handling (`errorHandler`).
- `server/src/controllers/`: Modular business logic for Authentication, Complaints, Technicians, and Admin operations.
- `server/src/utils/`: Haversine distance calculator (`haversine.js`) and standardized HTTP error envelopes (`errors.js`).

---

5. Database / ER Design (MongoDB Schemas)

5.1 `User` Collection

Field	Type	Constraints & Notes
`_id`	`ObjectId`	Primary Key
`name`	`String`	Required, trimmed, 2–100 characters
`email`	`String`	Required, unique, normalized lowercase
`phone`	`String`	Optional contact number
`passwordHash`	`String`	Required, `bcryptjs` salted hash (`select: false` by default)
`role`	`String`	Enum: `customer`, `technician`, `admin` (public registration blocks `admin`)
`isActive`	`Boolean`	Default `true`; deactivated accounts are blocked at login and token verification
`createdAt` / `updatedAt`	`Date`	Automatic Mongoose timestamps

5.2 `TechnicianProfile` Collection

Field	Type	Constraints & Notes
`_id`	`ObjectId`	Primary Key

Field	Type	Constraints & Notes
`userId`	`ObjectId`	Unique reference to `User._id`
`skills`	`[String]`	Array of supported fault types (e.g., `power_outage`, `meter_fault`)
`serviceAreas`	`[String]`	Array of covered localities (e.g., `Electronic City`, `Anekal`)
`isAvailable`	`Boolean`	Default `true`; controls inclusion in nearby discovery
`location`	`GeoJSON Point`	`{ type: 'Point', coordinates: [longitude, latitude] }` with sparse `2dsphere` index
`lastLocationAt`	`Date`	Timestamp of last coordinate update
`rating`	`Number`	1.0 to 5.0 rating indicator

5.3 `Complaint` Collection

Field	Type	Constraints & Notes
`_id`	`ObjectId`	Primary Key
`customerId`	`ObjectId`	Reference to `User._id` (indexed)
`problemType`	`String`	Enum: `power_outage`, `voltage_fluctuation`, `meter_fault`, `fuse_blown`, `billing_or_connection`,

Field	Type	Constraints & Notes
		`sparking_or_exposed_wire`, `other`
`description`	`String`	Required, 10–2000 characters
`address`	`String`	Optional manual address / landmark fallback
`location`	`GeoJSON Point`	Optional `{ type: 'Point', coordinates: [longitude, latitude] }`
`technicianId`	`ObjectId`	Nullable reference to assigned `User._id`
`status`	`String`	Enum: `pending`, `assigned`, `accepted`, `rejected`, `on_the_way`, `in_progress`, `resolved`, `cancelled`
`isHazardous`	`Boolean`	Automatically flagged when problem type or description indicates sparking/live wire risk
`statusHistory`	`[Object]`	Audit log entries `{ status, changedBy, note, timestamp }`

---

6. REST API Specification

All endpoints return `{ "success": true, "data": ... }` on success and `{ "success": false, "error": { "code": "...", "message": "..." } }` on error.

Method & Path	Access Role	Purpose & Behavior
`GET /api/health`	Public	Returns HTTP 200, service health, and MongoDB connection state
`POST /api/auth/register`	Public	Registers `customer` or `technician` (rejects `admin` role with HTTP 400)
`POST /api/auth/login`	Public	Verifies credentials with `bcryptjs` and issues 24h JWT
`GET /api/auth/me`	Authenticated	Returns current user profile and technician profile if applicable
`GET /api/complaints`	Role-based	Customers see own; Technicians see assigned + unassigned pending; Admins see all
`POST /api/complaints`	Customer	Creates complaint with optional GPS coordinates and hazard detection
`GET /api/complaints/:id`	Owner / Assigned Tech /...	Returns full complaint details and populated `statusHistory`
`PATCH /api/complaints/:id`	Role & State-based	Enforces valid state transitions (`accepted → on_the_way → in_progress → resolved`)
`DELETE /api/complaints/:id`	Owner (pending) / Admin	Soft-cancels complaint (`status: 'cancelled'`) and records audit trail

Method & Path	Access Role	Purpose & Behavior
`GET /api/technicians/nearby`	Authenticated	Ranks active available technicians by Haversine distance (`?lat=&lng=&radiusKm=`)
`PATCH /api/technicians/me/availability`	Technician	Toggles `isAvailable` status
`PATCH /api/technicians/me/location`	Technician	Updates technician GeoJSON `[lng, lat]` coordinates
`POST /api/complaints/:id/assign`	Admin / Owner	Assigns an active technician and sets status to `assigned`
`POST /api/complaints/:id/accept`	Assigned Technician	Accepts request; returns `409 ALREADY_ACCEPTED` on duplicate attempts
`POST /api/complaints/:id/reject`	Assigned Technician	Rejects request and returns complaint to `pending` queue for reassignment
`GET /api/admin/summary`	Admin	Returns system-wide user, technician, and complaint counters + directory lists
`PATCH /api/admin/users/:id`	Admin	Activates/deactivates user accounts or updates roles

---

7. Implementation & User Interface Workflows

- **Public Landing & Auth Pages:** Present clear scope disclaimers, electrical hazard warnings, registration forms, and one-click demo credential buttons on `/login`.
- **Customer Dashboard (`/customer`):** Features a complaint submission form with instant hazard warning banner, browser Geolocation API button (`navigator.geolocation.getCurrentPosition`), interactive Leaflet + OpenStreetMap map with click-to-pin fallback, nearby available technicians table sorted by straight-line Haversine distance, one-click "Request Nearest Tech" assignment, and pending complaint cancellation.
- **Technician Dashboard (`/technician`):** Provides an online/offline availability toggle, field GPS coordinate updater, and an actionable queue of assigned and eligible requests with `Accept`, `Reject`, `Mark On The Way`, `Start Repair`, and `Mark Resolved` workflow controls.
- **Admin Dashboard (`/admin`):** Displays real-time summary cards (total users, available technicians, pending/active/resolved complaints), status-filtered complaint management with technician assignment dropdowns, and user account activation/deactivation toggles.

---

8. Testing & Verification Results

The automated integration test suite (`server/tests/api.test.js` using Node's native test runner and supertest) verifies all acceptance criteria:

1. `GET /api/health` returns HTTP 200 OK.
2. Invalid registration/login payloads and unauthorized `admin` role self-registrations are rejected with HTTP 400 `VALIDATION_ERROR`.
3. Passwords are stored as `bcryptjs` hashes (`$2...`) and never exposed in API responses.
4. Missing or denied browser GPS coordinates gracefully fall back to manual street addresses (`HTTP 201`).
5. Customer B is blocked (`HTTP 403 FORBIDDEN`) from viewing or modifying Customer A's private complaints.
6. `GET /api/technicians/nearby` accurately computes Haversine distances and sorts the closest technician first.

7. Unassigned technicians are blocked (`HTTP 403`) from accepting another technician's assignment; duplicate acceptance returns `HTTP 409 ALREADY_ACCEPTED`.
8. Invalid state transitions (e.g., jumping directly from `accepted` to `resolved`) are rejected with `HTTP 400 INVALID_TRANSITION`, while sequential transitions (`accepted → on_the_way → in_progress → resolved`) succeed and log to `statusHistory`.
9. Non-admin users receive `HTTP 403 FORBIDDEN` on `/api/admin/*` endpoints.

---

9. Team Roles & Individual Contributions

- **Member 1 — React Frontend Lead:** Built Vite + React Router SPA architecture, `AuthContext`, responsive CSS design system, `MapPicker` Leaflet + OpenStreetMap integration, browser Geolocation fallback handling, and role dashboards (`CustomerDashboard`, `TechnicianDashboard`, `AdminDashboard`, `ComplaintDetailPage`).
- **Member 2 — Express Backend Lead:** Designed modular REST API routes and controllers (`auth`, `complaints`, `technicians`, `admin`), implemented Haversine distance ranking (`haversine.js`), state transition guards, and Zod request validation middleware.
- **Member 3 — MongoDB & Data Engineering Lead:** Authored Mongoose schemas (`User`, `TechnicianProfile`, `Complaint`), configured GeoJSON `[longitude, latitude] 2dsphere` indexes, audit trail `statusHistory` subdocuments, and deterministic database seed script (`seed.js`).
- **Member 4 — Security, Auth & QA Lead:** Implemented `bcryptjs` hashing, JWT authentication and RBAC middleware (`authenticate`, `requireRole`), hazard detection warnings, Supertest automated API test suite (`api.test.js`), and end-to-end demo verification.

---

10. Limitations & Future Scope

- **Current Prototype Limitations:** Straight-line Haversine distance does not account for road networks or traffic; notifications are in-app upon refresh

rather than push/SMS; prototype does not integrate with live government electricity board SCADA/OMS systems.

- **Future Enhancements:** Integrate OSRM/Mapbox road-routing APIs for driving distance and turn-by-turn ETA; add WebSocket (Socket.io) real-time push updates for status transitions; support photo attachments of faulty meters/poles via cloud object storage; add multilingual regional language support (Kannada, Tamil, Hindi).